计算机网络

第三章 数据链路层基础

授课教师：张新有

西南交通大学

计算机与人工智能学院
School of Computing and Artificial Intelligence

本章目标

- 了解数据链路层在网络体系结构中的位置及基本功能和服务
- 掌握差错检测和纠正的基本原理和典型的编码方法（核心内容）
- 掌握无错信道和有错信道上停等协议的设计和实现方法（核心内容）
- 理解停等协议的性能问题及滑动窗口协议的基本思想
- 掌握回退N和选择重传两种典型滑动窗口协议的工作机制（核心内容）
- 了解点到点链路层协议PPP与PPPoE（实验环节）

本章内容

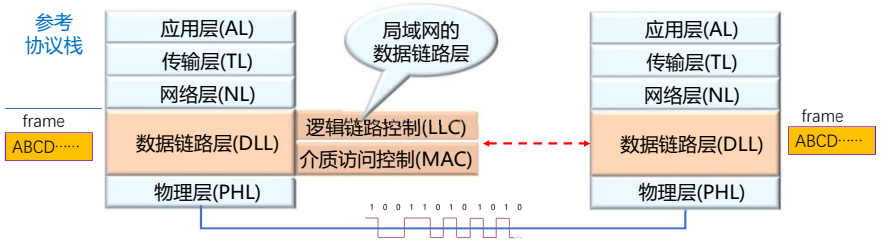
3.1 数据链路层的设计问题
3.2 差错检测和纠正（P21）
3.3 基本的数据链路层协议(P53)
3.4 滑动窗口协议(P69)
3.5 数据链路协议实例(P99)

1. 数据链路层在协议栈的位置
2. 数据链路层的功能
3. 数据链路层提供的服务
4. 成帧
5. 差错控制
6. 流量控制

1. 数据链路层在协议栈中的位置

数据链路层实体：

- 向下：利用物理层提供的bit流传输服务
- 向上：向网络层提供明确的 (well-defined) 服务接口



2. 数据链路层的功能

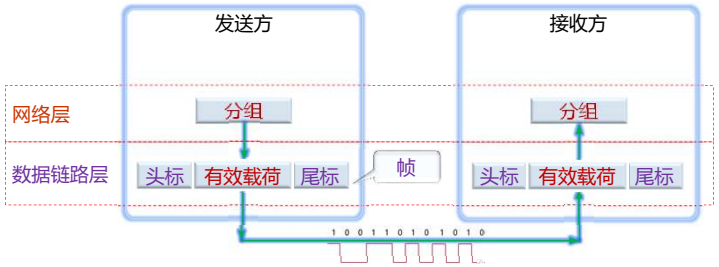
- 成帧 （Framing）
  - 成“帧”的主要目的是为了检测和纠正物理层在比特传输中可能出现的错误，数据链路层功能需借助“帧”的各个字段来实现可靠传输
- 差错控制 （Error Control）
  - 处理传输中出现的差错，如位错误、丢失等
- 流量控制 （Flow Control）
  - 确保发送方的发送速率，不大于接收方的处理速率
    - 避免接收缓冲区溢出

3. 数据链路层提供的服务

- 无确认 无连接 服务 （ Unacknowledged connectionless ）
  - 接收方不对收到的帧进行确认
  - 适用场景：误码率低的可靠信道；实时通信；
  - 网络实例：以太网
- 有确认 无连接 服务 （ Acknowledged connectionless ）
  - 每一帧都得到单独的确认
  - 适用场景：不可靠的信道（无线信道）
  - 网络实例：802.11
- 有确认 有连接 服务 （ Acknowledged connection-oriented ）
  - 适用场景：长延迟的不可靠信道

4. 成帧（Framing）

- 分组 (packet) 与 帧(frame)的关系

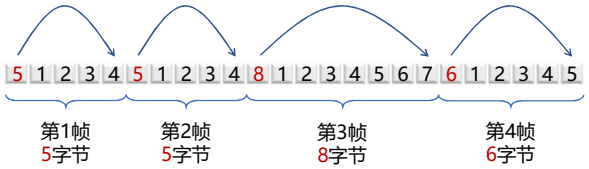


成帧（Framing）

- 关键问题：如何标识一个帧的开始？
  - 接收方必须能从物理层接收的比特流中明确区分出一帧的开始和结束，这个问题被称为帧同步或帧定界
  - 关键：选择何种定界符？定界符出现在数据部分如何处理？
- 成帧（framing）的方式
  1）字节计数法（ Byte count ）
  2）带字节填充的定界符法（ Flag bytes with byte stuffing ）
  3）带比特填充的定界符法（ Flag bits with bit stuffing ）
  4）物理层编码违例（ Physical layer coding violations ）

成帧（Framing）

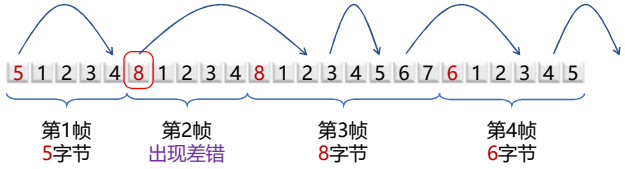
1）字节计数法：无差错传输的情形



问题：如果某个计数字节出错会发生什么情况？

成帧（Framing）

➢ 字节计数法：出现了一个字节差错的情形



破坏了帧的边界，导致一连串帧的错误

成帧（Framing）

2）带字节填充的定界符法

- 定界符（FLAG）：一个特殊的字节，比如 00000002，即 0x02，用于用于表示帧开始，用0x03表示帧的结束。

问题：如果有效载荷部分包含与“定界符”相同的字节会有什么问题？

成帧（Framing）

➢ 带字节填充的定界符法：典型代表是IBM二进制同步通信 (BSC) 协议。相关控制字符的ASCII定义：

符号名称	十进制	十六进制
SOH（start of headline 标题开始）	1	01
STX（start of text 正文开始）	2	02
ETX（end of text 正文结束）	3	03
EOT（end of transmission传输结束）	4	04
ENQ（enquiry 请求）	5	05
ACK（acknowledge 收到通知）	6	06
DLE（data link escape 数据链路转义）	16	10
NAK（negative acknowledge 拒绝接收）	21	15
SYN（(synchronous idle 同步空闲）	22	16
ETB（end of trans. block 传输块结束）	23	17

特点 是一次传送若干个字符组成的数据块而非一个字符，并规定10个字符作为该数据块的开头与结束标志及整个传输过程的控制信息(通信控制字)。由于被传送的数据块是由字符组成，故被称作面向字符的协议。

成帧（Framing）

- ASCII字符：帧头DLE STX　　帧尾DLE ETX
- DLE (Data Link Escape); STX (Start of Text); EXT (End of Text)
- 如果字符中出现DLE，就添一个DLE，称为字符填充。
- 缺点：完全依赖8位字符，特别是ASCII字符。

例：如果网络层发出：

A	DLE	B	C

经数据链路层填充：

DLE	STX	A	DLE	DLE	B	C	DLE	ETX

接收方网络层：

A	DLE	B	C

ASCII字符表

二进制	十进制	十六进制	符号	解释
0000 0000	0	0	NUL	空字符
0000 0001	1	1	SOH	标题开始
0000 0010	2	2	STX	正文开始
0000 0011	3	3	ETX	正文结束
0000 0100	4	4	EOT	传输结束
0000 0101	5	5	ENQ	询问
0000 0110	6	6	ACK	收到通知
0000 0111	7	7	BEL	铃
0000 1000	8	8	BS	退格
0000 1001	9	9	HT	水平制表符
0000 1010	10	0A	LF	换行键
0000 1011	11	0B	VT	垂直制表符
0000 1100	12	0C	FF	换页键
0000 1101	13	0D	CR	回车键
0000 1110	14	0E	SO	移出
0000 1111	15	0F	SI	移入
0001 0000	16	10	DLE	数据链路转义
0001 0001	17	11	DC1	设备控制1
0001 0010	18	12	DC2	设备控制2
0001 0011	19	13	DC3	设备控制3
0001 0100	20	14	DC4	设备控制4
0001 0101	21	15	NAK	拒绝接收
0001 0110	22	16	SYN	同步空闲
0001 0111	23	17	ETB	传输块结束
0001 1000	24	18	CAN	取消
0001 1001	25	19	EM	介质中断
0001 1010	26	1A	SUB	替换
0001 1011	27	1B	ESC	换码符
0001 1100	28	1C	FS	文件分隔符
0001 1101	29	1D	GS	组分隔符
0001 1110	30	1E	RS	记录分隔符
0001 1111	31	1F	US	单元分隔符
0111 1111	127	7F	DEL	删除

二进制	十进制	十六进制	符号
0010 0000	32	20	空格
0010 0001	33	21	!
0010 0010	34	22	"
0010 0011	35	23	#
0010 0100	36	24	$
0010 0101	37	25	%
0010 0110	38	26	&
0010 0111	39	27	'
0010 1000	40	28	(
0010 1001	41	29	)
0010 1010	42	2A	*
0010 1011	43	2B	+
0010 1100	44	2C	,
0010 1101	45	2D	-
0010 1110	46	2E	.
0010 1111	47	2F	/
0011 0000	48	30	0
0011 0001	49	31	1
0011 0010	50	32	2
0011 0011	51	33	3
0011 0100	52	34	4
0011 0101	53	35	5
0011 0110	54	36	6
0011 0111	55	37	7
0011 1000	56	38	8
0011 1001	57	39	9
0011 1010	58	3A	:
0011 1011	59	3B	;
0011 1100	60	3C	<
0011 1101	61	3D	=
0011 1110	62	3E	>
0011 1111	63	3F	?

二进制	十进制	十六进制	符号
0100 0000	64	40	@
0100 0001	65	41	A
0100 0010	66	42	B
0100 0011	67	43	C
0100 0100	68	44	D
0100 0101	69	45	E
0100 0110	70	46	F
0100 0111	71	47	G
0100 1000	72	48	H
0100 1001	73	49	I
0100 1010	74	4A	J
0100 1011	75	4B	K
0100 1100	76	4C	L
0100 1101	77	4D	M
0100 1110	78	4E	N
0100 1111	79	4F	O
0101 0000	80	50	P
0101 0001	81	51	Q
0101 0010	82	52	R
0101 0011	83	53	S
0101 0100	84	54	T
0101 0101	85	55	U
0101 0110	86	56	V
0101 0111	87	57	W
0101 1000	88	58	X
0101 1001	89	59	Y
0101 1010	90	5A	Z
0101 1011	91	5B	[
0101 1100	92	5C	\
0101 1101	93	5D	]
0101 1110	94	5E	^
0101 1111	95	5F	_

二进制	十进制	十六进制	符号
0110 0000	96	60	`
0110 0001	97	61	a
0110 0010	98	62	b
0110 0011	99	63	c
0110 0100	100	64	d
0110 0101	101	65	e
0110 0110	102	66	f
0110 0111	103	67	g
0110 1000	104	68	h
0110 1001	105	69	i
0110 1010	106	6A	g
0110 1011	107	6B	k
0110 1100	108	6C	l
0110 1101	109	6D	m
0110 1110	110	6E	n
0110 1111	111	6F	o
0111 0000	112	70	p
0111 0001	113	71	q
0111 0010	114	72	r
0111 0011	115	73	s
0111 0100	116	74	t
0111 0101	117	75	u
0111 0110	118	76	v
0111 0111	119	77	w
0111 1000	120	78	x
0111 1001	121	79	y
0111 1010	122	7A	z
0111 1011	123	7B	{
0111 1100	124	7C	|
0111 1101	125	7D	}
0111 1110	126	7E	~

成帧（Framing）

3）带**比特**填充的定界符法

- 定界符：两个0比特之间，连续6个1比特，即01111110，0x7E

0 1 0 1 1 0 1 1 1 1 1 1 0 0 1 1 1 0 0 0 1 0 1 1

上一帧　　定界符　　下一帧

问题：如果有效载荷部分包含与“定界符”相同的位组合如何解决？

成帧（Framing）

➢ 带**比特**填充的定界符法：发送方检查有效载荷

- 若在有效载荷中出现连续5个1比特，则直接插入1个0比特

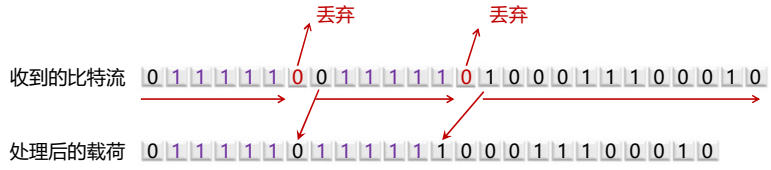
原始载荷　0 1 1 1 1 1 0 1 1 1 1 1 1 0 0 0 1 1 1 0 0 0 1 0

载荷填充　0 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 0 0 0 1 0

成帧（Framing）

➢ 带**比特**填充的定界符法：接收方的处理

- 若出现连续5个1比特
  - 若下一比特为0，则为有效载荷，直接丢弃0比特
  - 若下一比特为1，则连同后一比特的0，构成定界符，一帧结束



成帧（Framing）

4）物理层编码违例：

- 核心思想：选择的定界符不会在数据部分出现
- 4B/5B编码方案
  - 4比特数据映射成5比特编码，剩余的一半码字（16个码字）未使用，可以用做帧定界符
  - 例如： 00110组合不包含在4B/5B编码中，可做帧定界符
- 前导码
  - 存在很长的 前导码（preamble），可以用作定界符
  - 例如：传统以太网、802.11
- 曼切斯特编码 / 差分曼切斯特编码
  - 正常的信号在周期中间有跳变，持续的高电平（或低电平）为违例码，可以用作定界符
  - 例如：802.5令牌环网

5. 差错控制

➢ 链路层存在的一个问题：信道的噪声导致数据传输问题

- 差错（incorrect）：数据发生错误
- 丢失（lost）：接收方未收到
- 乱序（out of order）：先发后到，后发先到
- 重复（repeatedly delivery）：一次发送，多次接收

➢ 解决方案：差错检测与纠正、确认重传

- 确认：接收方校验数据（差错校验），并给发送方应答，防止差错
- 定时器：发送方启动定时器，防止丢失
- 顺序号：接收方检查序号，防止乱序递交、重复递交

6. 流量控制

➢ 链路层存在的另一个问题：接收方的处理速率

- 接收方的接收缓冲区溢出

➢ 解决方案

- 基于反馈 (feedback-based) 的流量控制
  接收方反馈，发送方调整发送速率
- 基于速率 (rate-based) 的流量控制
  发送方根据内建机制，自行限速

本章内容

3.1 数据链路层的设计问题
3.2 差错检测和纠正
3.3 基本的数据链路层协议(P53)
3.4 滑动窗口协议(P69)
3.5 数据链路协议实例(P99)

1. 差错检测与纠正概述
2. 典型的纠错码
3. 典型的检错码

1. 差错检测和纠正概述

➢ 如何解决信道传输差错问题

- 通常采用增加冗余信息（或称校验信息）的策略
- 简单示例：每个比特传3份，如果每比特的3份中有一位出错，可以纠正

0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1
0	1	0	0	1	0	1	0	1	0	1	1	0	1

携带2/3的冗余信息！
冗余信息量大！

差错检测和纠正概述

➢ 目标：保证一定检错和纠错能力前提下，如何减少冗余信息量？

➢ 考虑的问题

- 信道的特征和传输质量需求
- 冗余信息的计算方法、携带的冗余信息量
- 算法的计算复杂度等

➢ 两种主要策略

- 检错码（error-detecting code）
- 纠错码（error-correcting code）

差错检测和纠正概述

➢ 检错码（error-detecting code）

- 在被发送的数据块中，包含一些冗余信息，但这些信息只能使接收方推断是否发生错误，而不能推断哪位发生错误，接收方可以通知发送方重传数据；
- ARQ协议，即自动重传请求（Automatic Repeat-reQuest），是OSI模型中的错误纠正协议之一。它通过使用确认和重传这两个机制，在不可靠服务的基础上实现可靠的信息传输。也称反馈纠错。
- 主要用在高可靠、误码率较低的信道上，例如光纤链路；
- 偶尔发生的差错，可以通过重传解决差错问题；

差错检测和纠正概述

- 纠错码（error-correcting code）
  - 发送方在每个数据块中加入足够的冗余信息，使得接收方能够判断接收到的数据是否有错，并能纠正错误（定位出错的位置）
  - 主要用于错误发生比较频繁的信道上，如无线链路
  - 经常用于物理层，也用于高层（例如，实时流媒体应用和内容分发）
  - 使用纠错码的技术通常称为前向纠错（FEC，Forward Error Correction）
  - 混合纠错HEC的方式是：少量纠错在接收端自动纠正；差错较严重，超出自行纠正能力时，就向发信端发出询问信号，要求重发。因此，“混合纠错HEC”是“前向纠错FEC”及“反馈纠错ARQ”两种方式的混合。

差错检测和纠正概述

- 码字 (code word)：一个包含m个数据位和r个校验位的n位单元
  - 描述为 (n, m) 码，n=m+r
- 码率 (code rate)：码字中不含冗余部分所占的比例，可用m/n表示
- 海明距离 (Hamming distance)：两个码字之间不同对应比特的数目
  - 例：0000000000 与0000011111的海明距离为5
  - 若两个码字的海明距离为d，则d个比特错就可把一个码字转换成另一个码字
  - 为了检查出d个错（比特错），可以使用海明距离为 **d+1** 的编码
  - 为了纠正d个错，可以使用海明距离为 **2d+1** 的编码

差错检测和纠正概述

- 例如：
  - 一个10bits，但只有4个有效码字的编码方案：0000000000, 0000011111, 1111100000, 1111111111
  - 海明距离为5，可以检测4位错，纠正2位错
  - 如果已知只有1位或2位错误，接收方接收0000000111，可纠错
    - 则可知原码字为：0000011111
  - 如果发生3位错误，例如0000000000变成0000000111，接收方无法纠正错误，但可以检测出错误

2. 典型检错码

- 常用的检错码包括：

  1) 奇偶检验 (Parity Check)：1位奇偶校验，最简单、最基础的检错码

  2) 校验和 (Checksum)：主要用于TCP/IP体系中的网络层和传输层

  3) 循环冗余校验 (Cyclic Redundancy Check，CRC)：数据链路层广泛使用的校验方法

1) 典型检错码—奇偶校验

➢ 1位奇偶校验：增加1位校验位，可以检查奇数位错误

- 偶校验：保证1的个数为偶数个，例如：

1	0	1	1	0	1	0	0

数据位　　校验位

- 奇校验：保证1的个数为奇数个，例如：

1	0	1	1	0	1	0	1

数据位　　校验位

考虑二维奇偶校验，是否可以进行纠错？

2) 典型检错码—校验和

➢ TCP/IP体系中采用的校验方法

发送方：进行 16 位二进制补码求和运算，计算结果取反，随数据一同发送

数据　1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
　　1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

　　1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

校验和　1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
　　0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

接收方：进行 16 位二进制补码求和运算（包含校验和），结果非全1，则检测到错误

1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0　数据
1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1　校验和

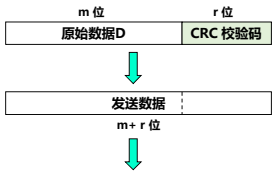
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

未检测到错误

3) 典型检错码—循环冗余校验CRC

➢ CRC校验码计算方法

- 设原始数据D为m位二进制位模式
- 如果要产生r位CRC校验码，事先选定一个r+1位二进制位模式G (称为生成多项式G(x)，收发双方提前商定)，G的最高位为1
- 将原始数据D乘以2^r（相当于在D后面添加 r 个0），产生m+r位二进制位模式，用G对该位模式做模2除，得到余数R（r位，不足r位前面用0补齐）即为CRC校验码



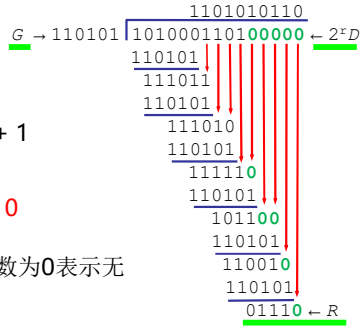
CRC校验能力：能检测出少于r+1位的突发错误

接收端如何计算？

检错码—循环冗余校验CRC

➢ CRC校验码计算示例

- D = 1010001101
- r = 5
- G = 110101 或 G = $x^5 + x^4 + x^2 + 1$
- R = 01110
- 实际传输数据：101000110101110



▪解码：收方收到码“模2除”以G，余数为0表示无错，否则，有错。

检错码—循环冗余校验CRC

- 四个国际标准生成多项式G(x)
  - CRC-12 = $x^{12}+x^{11}+x^3+x^2+x+1$
  - CRC-16 = $x^{16}+x^{15}+x^2+1$
  - CRC-CCITT = $x^{16}+x^{12}+x^5+1$
  - CRC-32 = $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$

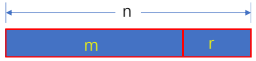
CRC特点：检错能力强，开销小，易于用编码器及检测电路实现。

检错能力：它所不能发现的错误的几率仅为0.0047%以下。

性能和开销：均远远优于奇偶校验及算术和校验等方式。

CRC在数据存储和数据通讯领域CRC无处不在：通讯协议X.25的FCS采用CRC-CCITT; WinRAR、NERO、ARJ、LHA等压缩工具软件采用的是CRC-32; 磁盘驱动器读写采用CRC-16; 图像存储格式GIF、TIFF等也都用CRC作为检错手段。

3. 典型纠错码



- 设计纠错码（n，m）
  - 要求：m个信息位，r个校验位，纠正1比特错 n=m+r
  - 对2^m个有效信息中任何一个，有n个与其距离为1的无效码字：
    - 可以考虑将该信息对应的合法码字的n位逐个取反，得到n个距离为1的非法码字，需要n+1个位模式来标识
  - 因此有：$(n + 1)\ 2^m \le 2^n$
  - 利用 n = m + r，得到 $(m + r + 1) \le 2^r$
  - 在给定m的情况下，用该式可以得出纠正单比特错误校验位数r的下界，

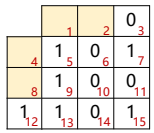
例如，若m=11，则r最小为4，n=15;

1) 典型纠错码—海明码

- 目标：以奇偶校验为基础，如何找到出错位置，提供1位纠错能力
- 理解海明码编码过程，以 (15, 11)海明码为例
  - 例如：11比特的数据0101 1001 101
  - 11比特数据按顺序放入数据位
  - 校验位：2的幂次方位，记为：

    $p(1)=p(2^0)$, $p(2)=p(2^1)$, $p(4)=p(2^2)$, $p(8)=p(2^3)$
  - 每个校验位对数据位的子集做校验，缩小错误定位范围
  - 问题：每个校验位如何计算？

r_1 r_2 m_3 r_4 m_5 m_6 m_7 r_8 m_9 m_{10} m_{11} m_{12} m_{13} m_{14} m_{15}



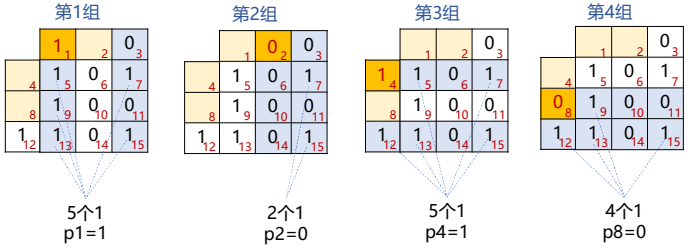
校验位 数据位

右下角数字为码字中位序号

典型纠错码—海明码

- 子集的选择与校验位计算
  - 海明码缺省为偶校验（也可以使用奇校验）

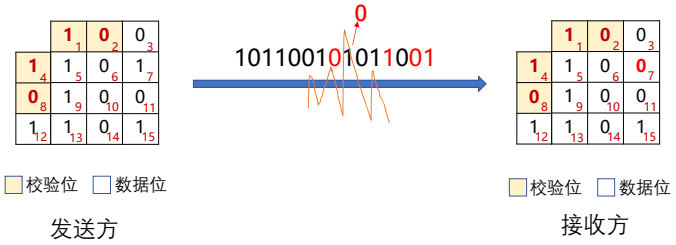
每组的数据位 每组的校验位



典型纠错码—海明码

➢ 码字的发送与接收

- 如果发送过程中第7位出现差错，如何定位错误？

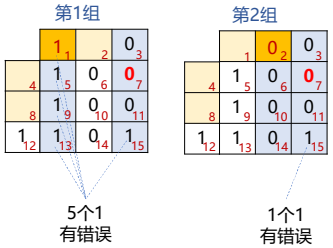


典型纠错码—海明码

➢ 定位错误与纠正

- 组1和组2的校验结果可以定位错误所在的列
- 例如：组1和组2校验结果都指明存在错误，可定位错误位于第4列
- 其他出错情况判断

出错列	2	3	4
组1	×	√	×
组2	√	×	×



典型纠错码—海明码

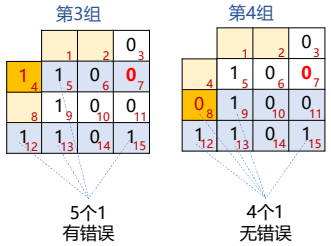
➢ 定位错误与纠正（续）

- 组3和组4的校验结果可以定位错误所在的行
- 例如：组3校验结果指明存在错误，组4校验结果指明无错误，可定位错误位于第2行

➡ 结论：错误位于第2行、第4列（位置7）

- 其他出错情况判断

出错行	2	3	4
组3	×	√	×
组4	√	×	×



典型纠错码—海明码

➢ 定位错误与纠正（续）

- 总体的定位错误列表

出错位	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
组1	×	√	×	√	×	√	×	√	×	√	×	√	×	√	×
组2	√	×	×	√	√	×	×	√	√	×	×	√	√	×	×
组3	√	√	√	×	×	×	×	√	√	√	√	×	×	×	×
组4	√	√	√	√	√	√	√	×	×	×	×	×	×	×	×

典型纠错码—海明码

- 海明码纠正的实现过程
  - 每个码字到来前，接收方计数器C清零
  - 接收方检查每个校验位k的奇偶值是否正确(每组运算)
  - 若pk奇偶值不对，计数器加 k
  - 所有校验位检查完后，若计数器值为0，则码字有效；若计数器值为 j，则第 j 位出错。

  例：若校验位p1、p2、p8出错，则第11位变反。

编码形式：

r1 r2 m3 r4 m5 m6 m7 r8 m9 m10 m11 …

处理方式

发方：

- 编码：r_i = 各相关信息位的异或

收方：

- 解码：设计数器C=0，
  - r_k与相关信息位的异或为0，C=C+0
  - r_k与相关信息位的异或为1，C=C+k
  - C为出错位置。

海明码纠正突发性错

- 使用海明码纠正突发错误
  - 可采用k个码字(n=m+r)组成 k×n矩阵，按列发送，接收方恢复成k×n矩阵
  - kr个校验位，km个数据位，可纠正最多为k个的突发性连续比特错

码字(n位)

K个码字

m_{11}	m_{12}	… …	m_{1n}
m_{21}	m_{22}	… …	m_{2n}
… …			
m_{k1}	m_{k2}	… …	m_{kn}

位传输顺序

例：

$$r_1 \oplus m_3 \oplus m_5 \oplus m_7$$
$$r_2 \oplus m_3 \oplus m_6 \oplus m_7$$
$$r_4 \oplus m_5 \oplus m_6 \oplus m_7$$

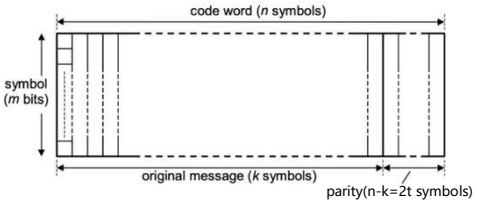
位传输顺序

字符码	ASCII	校验
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	11111001111

2) 典型纠错码—Reed-Solomon code

- RS码：以有限域运算为基础，提供多位纠错能力
  - RS会将需要编码的流数据重新排列为以Symbol为单位的数据块
  - m表示符号的大小，如 m=4 表示每个符号由4位二进制数组成
  - 对于一个(n, k) RS编码，k为原始数据符号数，n-k为校验符号数
  - n-k=2t，t表示能够纠正的错误数

Reed-Solomon code:
里德-所罗门码(又称里所码)
一种FEC。



典型纠错码—Reed-Solomon code

- 以200/400GBASE-R使用的RS(544,514)编码为例：
  - 每个Symbol为10位二进制数 (m=10)
  - 原始需要保护的数据为514个symbols (k=514)
  - 校验数据为30个symbols (2t=30)
  - 最终完成的编码为544个symbols (n=544)
  - 可以纠正15个symbols的错误

典型纠错码—Reed-Solomon code

➢ IEEE 802.3 119.2.4.6中定义的RS编码算法:

生成多项式

$$g(x)=\prod_{j=0}^{2t-1}(x-\alpha^j)$$

$$(m_{k-1}x^{k-1}+\cdots+m_1x+m_0)\times x^{2t}$$

$$m_{k-1}x^{n-1}+\cdots+m_1x^{2t+1}+m_0x^{2t}+0x^{2t-1}+\cdots+0x+0 \quad \% \quad g_{2t}x^{2t}+\cdots+g_1x+g_0 \quad =$$

$$p_{2t-1}x^{2t-1}+\cdots+p_1x+p0$$

$$m_{k-1}x^{n-1}+\cdots+m_1x^{2t+1}+m_0x^{2t}+p_{2t-1}x^{2t-1}+\cdots+p_1x+p_0$$

Code word symbols

Message symbols　　Parity symbols

典型纠错码—Reed-Solomon code

➢ 以 (15, 11) RS码为例，理解RS码的编码过程

- 每个Symbol为4位二进制数 (m=4，相当一个16进制值)
- 原始数据为11个Symbol (k=11)，校验数据为4个symbols (2t=4)，
- 完成的编码为15个symbols (n=15)
- 假设原始数据为[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]（每个为一个symbol）
- 后面补4个0，即[1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11, 0, 0, 0, 0]， 并以多项式形式表示
- 除以生成多项式：$g(x)=\prod_{j=0}^{3}(x-2^j)=(x-1)(x-2)(x-4)(x-8)$ $=x^4+15x^3+3x^2+x+12$
- 取最终余数[3, 3, 12, 12]加到补过0的原始数据中
- 生成最终编码[1, 2, 3, 4, 5, 6, 7, 8, 9,10,11, 3, 3,12,12]，可以用多项式表达，记为C(x)

注意以上运算均为有限域GF(2^4)中的运算，本原多项式为 x^4+x+1

典型纠错码—Reed-Solomon code

➢ 校验码的具体计算过程：

- 类似于CRC校验码的生成过程
- 运算为有限域GF(2^4)中的运算

	x^{14}	x^{13}	x^{12}	x^{11}	x^{10}	x^9	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
	1	2	3	4	5	6	7	8	9	10	11	0	0	0	0
$\times x^{10}$	1	15	3	1	12										
		13	0	5	9	6									
$\times 13x^9$		13	7	4	13	3									
			7	1	4	5	7								
$\times 7x^8$			7	11	9	7	2								
				10	13	2	5	8							
$\times 10x^7$				10	12	13	10	1							
					1	15	15	9	9						
$\times 1x^6$					1	15	3	1	12						
						0	12	8	5	10					
$\times 0x^5$						0	0	0	0	0					
							12	8	5	10	11				
$\times 12x^4$							12	8	7	12	15				
								0	2	6	4	0			
$\times 0x^3$								0	0	0	0	0			
									2	6	4	0	0		
$\times 2x^2$									2	13	6	2	11		
										11	2	2	11	0	
$\times 11x$										11	3	14	11	13	
											1	12	0	13	0
$\times 1$											1	15	3	1	12
												3	3	12	12

典型纠错码—Reed-Solomon code

➢ 理解RS码的解码过程

- 根据编码的生成过程可知，若传输过程中没有任何错误，那么接收到的编码数据块去除生成多项式 g(x) 是可以整除的，没有余数的，$C(\alpha^i)=0$

$$m_{k-1}x^{n-1}+\cdots+m_1x^{2t+1}+m_0x^{2t}+p_{2t-1}x^{2t-1}+\cdots+p_1x+p_0 \quad \% \quad \prod_{j=0}^{2t-1}(x-\alpha^j) \quad =0$$

Message symbols　　Parity symbols

$$C(x)=c_{n-1}x^{n-1}+\cdots+c_1x+c_0$$

编码数据的多项式表示

- 如果传输过程中有错误，接收数据是原始编码数据叠加上错误数据，这时将 α^i 代入，结果不为0，这个结果被称作「Syndrome」，该结果只与引入的错误有关，如果能获取错误出现的位置和值，就可以纠正错误

典型纠错码—Reed-Solomon code

西南交通大学 Southwest Jiaotong University

- 第1步：根据收到的码字，计算Syndromes值
  - 若计算出的Syndromes全为0，则判断码字没有发生错误，直接提取message symbols
  - 若计算出的Syndromes不为0，则表示收到的码字发生了错误
  - 本例中 $g(x)=\prod_{j=0}^{3}(x-2^j)=(x-1)(x-2)(x-4)(x-8)$ = $x^4+15x^3+3x^2+x+12$

$$\begin{bmatrix} 1^0 & 2^0 & 4^0 & 8^0 \\ 1^1 & 2^1 & 4^1 & 8^1 \\ \vdots & \vdots & \vdots & \vdots \\ 1^{14} & 2^{14} & 4^{14} & 8^{14} \end{bmatrix}$$

所有运算均为有限域GF(2^4)中的运算

errors

[12 12 7 3 11 10 9 8 7 0 5 4 3 2 1]

received codeword

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 4 & 3 & 12 \\ 1 & 8 & 12 & 10 \\ 1 & 3 & 5 & 15 \\ 1 & 6 & 7 & 1 \\ 1 & 12 & 15 & 8 \\ 1 & 11 & 9 & 12 \\ 1 & 5 & 2 & 10 \\ 1 & 10 & 8 & 15 \\ 1 & 7 & 6 & 1 \\ 1 & 14 & 11 & 8 \\ 1 & 15 & 10 & 12 \\ 1 & 13 & 14 & 10 \\ 1 & 9 & 13 & 15 \end{bmatrix} = [2\ \ 10\ \ 9\ \ 1]$$

Syndromes

parity check matrix

计算中使用的parity check matrix，与除g(x)效果相同

典型纠错码—Reed-Solomon code

西南交通大学 Southwest Jiaotong University

- GF(2^4)含有16个元素，本原多项式为P(x)=x^4+x+1，除了 0 、1外，另外14个符号均由本原多项式生成：

生成元素	多项式表示	二进制表示	数值表示	推导过程
0	0	0000	0	
x^0	x^0	0001	1	
x^1	x^1	0010	2	
x^2	x^2	0100	4	
x^3	x^3	1000	8	
x^4	x+1	0011	3	x^3*x = x^4 mod P(x) = x+1
x^5	x^2+x	0110	6	x^4*x = (x+1)*x = x^2+x
x^6	x^3+x^2	1100	12	
x^7	x^3+x+1	1011	11	x^6*x = (x^3+x^2)*x = x^4 +x^3 mod P(x) = x^3+x+1
x^8	x^2+1	0101	5	
x^9	x^3+x	1010	10	
x^10	x^2+x+1	0111	7	x^9*x=(x^3+x)*x = x^4+x^2 mod P(x) = x^2+x+1
x^11	x^3+x^2+x	1110	14	
x^12	x^3+x^2+x+1	1111	15	x^11*x=(x^3+x^2+x)*x = x^4+x^3+x^2 mod P(x) = x^3+x^2+x+1
x^13	x^3+x^2+1	1101	13	x^12*x=(x^3+x^2+1)*x = x^4+x^3+x mod P(x)= x^3+1
x^14	x^3+1	1001	9	x^13*x=(x^3+x^2+1)*x = x^4+x^3+x mod P(x) = x^3+1
x^15	1	0001	1	x^14*x = (x^3+1)*x = x^4+x mod P(x) = 1

典型纠错码—Reed-Solomon code

西南交通大学 Southwest Jiaotong University

- GF(2^4)的乘法表：

*	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	2	4	6	8	10	12	14	3	1	7	5	11	9	15	13
3	3	6	5	12	15	10	9	11	8	13	14	7	4	1	2
4	4	8	12	3	7	11	15	6	2	14	10	5	1	13	9
5	5	10	15	7	2	13	8	14	11	4	1	9	12	3	6
6	6	12	10	11	13	7	1	5	3	9	15	14	8	2	4
7	7	14	9	15	8	1	6	13	10	3	4	2	5	12	11
8	8	3	11	6	14	5	13	12	4	15	7	10	2	9	1
9	9	1	8	2	11	3	10	4	13	5	12	6	15	7	14
10	10	7	13	14	4	9	3	15	5	8	2	1	11	6	12
11	11	5	14	10	1	15	4	7	12	2	9	13	6	8	3
12	12	11	7	5	9	14	2	10	6	1	13	15	3	4	8
13	13	9	4	1	12	8	5	2	15	11	6	3	14	10	7
14	14	15	1	13	3	2	12	9	7	6	8	4	10	11	5
15	15	13	2	9	6	4	11	1	14	12	3	8	7	5	10

典型纠错码—Reed-Solomon code

西南交通大学 Southwest Jiaotong University

- 第2步：定位错误的位置和值
  - 如果传输过程中有错，则接收数据就是原始数据叠加上错误数据，即

$$\boldsymbol{R}(\boldsymbol{\alpha}^i)=\boldsymbol{C}(\boldsymbol{\alpha}^i)+\boldsymbol{E}(\boldsymbol{\alpha}^i)=\boldsymbol{0}+\boldsymbol{E}(\boldsymbol{\alpha}^i)=\boldsymbol{S_i}$$

  - 假设有v 个错误，则syndrome可以重写为：

$$S_i=E(\alpha^i)$$
$$=Y_1\alpha^{ie_1}+Y_2\alpha^{ie_2}+\cdots+Y_v\alpha^{ie_v}$$
$$=Y_1X_1^i+Y_2X_2^i+\cdots+Y_vX_v^i=\sum_{j=1}^{v}Y_jX_j^i$$

  - 定义一个多项式

$$\Lambda(x)=(1+X_1x)(1+X_2x)\cdots(1+X_vx)$$
$$=1+\Lambda_1x+\Lambda_2x^2+\cdots+\Lambda_vx^v$$

  - 得到：$0=S_{i+v}+\Lambda_1S_{i+v-1}+\cdots+\Lambda_vS_i$

本例中，有4个校验符号，因此能纠正2个错误，其中X_1，X_2表示错误的位置，Y_1，Y_2表示错误的值

$$0=S_{i+v}+\Lambda_1S_{i+v-1}+\cdots+\Lambda_vS_i \qquad v=2$$

$$0=S_2+\Lambda_1S_1+\Lambda_2S_0$$
$$0=S_3+\Lambda_1S_2+\Lambda_2S_1$$

$S_0=2$, $S_1=10$, $S_2=9$, $S_3=1$

$$9=10\Lambda_1+2\Lambda_2$$
$$1=9\Lambda_1+10\Lambda_2$$

$$\Lambda_1=14$$
$$\Lambda_2=14$$

$$\Lambda(x)=1+14x+14x^2=(1+4x)(1+10x)$$

$$2=Y_1+Y_2$$
$$10=4Y_1+10Y_2$$

$$Y_1=4$$
$$Y_2=6$$

$$S_i=Y_14^i+Y_210^i$$

$X_1=4$, $X_2=10$

$e_1=2$, $e_2=9$

$$E(x)=Y_1x^{e_1}+Y_2x^{e_2}=4x^2+6x^9$$

错误位置的定位为位置2和位置9，错误值为4和6

红色箭头为运算主线

典型纠错码—Reed-Solomon code

- 第3步：恢复原始信息
  - 将错误信息E(x)加到接收消息R(x)上，即可纠正错误信息
  - 删除冗余信息，即可恢复原始数据

$$E(x) = 6x^9 + 4x^2$$

$$R(x) = 1x^{14} + 2x^{13} + 3x^{12} + 4x^{11} + 5x^{10} + \boxed{0x^9} + 7x^8 + 8x^7 + 9x^6 + 10x^5 + 11x^4 + 3x^3 + \boxed{7x^2} + 12x + 12$$

$$C(x) = 1x^{14} + 2x^{13} + 3x^{12} + 4x^{11} + 5x^{10} + 6x^9 + 7x^8 + 8x^7 + 9x^6 + 10x^5 + 11x^4 + \cancel{3x^3 + 3x^2 + 12x + 12}$$

[1 2 3 4 5 6 7 8 9 10 11] 原始数据

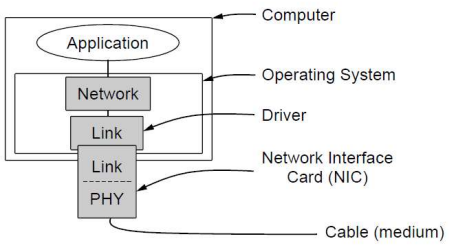
本章内容

3.1 数据链路层的设计问题
3.2 差错检测和纠正（P21）
3.3 基本的数据链路层协议
3.4 滑动窗口协议（P69）
3.5 数据链路协议实例（P99）

1. 假设与定义
2. 乌托邦式单工协议
3. 无错信道单工停止-等待协议
4. 有错信道单工停止-等待协议

1. 假设与定义

- 物理层进程和某些数据链路层进程部分运行在专用硬件上（网络接口卡）
- 数据链路层进程的其他部分和网络层进程作为操作系统的一部分运行在CPU上，数据链路层进程的软件通常以设备驱动的形式存在

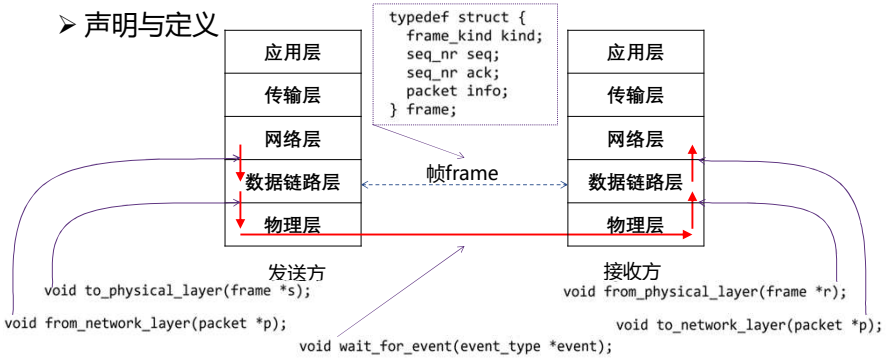


关键假设

- 分层进程独立假设
  - 网络层、数据链路层、物理层为独立的进程
  - 进程间通过传递消息实现通信
- 提供可靠服务假设
  - 提供可靠的、面向连接的服务
  - 数据链路层发送的数据随时可从网络层获得
- 只处理通信错误假设
  - 仅处理通信错误
  - 假设机器不会崩溃，不考虑断电、重启等引起的问题

基本的协议定义

➢ 声明与定义



event_type: 帧到达frame_arrival、帧出错cksum_err、超时timeout等

2. 乌托邦式单工协议(1)

➢ 假设
- 单工（Simplex）协议：数据单向传输
- 完美信道：帧不会丢失或受损
- 始终就绪：发送方/接收方的网络层始终处于就绪状态
- 瞬间完成：发送方/接收方能够生成/处理无穷多的数据

➢ 乌托邦：完美但不现实的协议
- 不处理任何流量控制或纠错工作
- 接近于无确认的无连接服务，必须依赖更高层次解决上述问题

乌托邦式单工协议(2)

➢发送方
- 在循环中不停发送
- 从网络层获得数据
- 封装成帧
- 交给物理层
- 完成一次发送

```
frame s;
packet buffer;

while (true) {
        from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
}
```

➢ 接收方
- 在循环中持续接收
- 等待帧到达（frame_arrival）
- 从物理层获得帧
- 解封装，将帧中的数据传递给网络层
- 完成一次接收

```
frame r;
event_type event;

while (true) {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
}
```


3. 无错信道上的停等式协议(1)

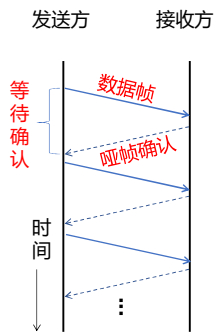
➢ 不再假设
- 接收方能够处理以无限高速进来的数据
- 发送方以高于接收方能处理到达帧的速度发送帧，导致接收方被“淹没”(overwhelming)

➢ 仍然假设
- 通信信道不会出错（Error-Free）
- 数据传输保持单向，但是需要双向传输链路（半双工物理信道）

无错信道上的停等式协议(2)

➢ 停-等式协议（stop-and-wait）

- 发送方发送一帧后暂停，等待确认（Acknowldgement）到达后发送下一帧
- 接收方完成接收后，回复确认接收.
- 确认帧的内容是不重要的：哑帧（dummy frame）



无错信道上的停等式协议(3)

➢ 发送方

- 完成一帧发送后
- 等待确认到达
- 确认到达后，发送下一帧

```
while (true) {
        from_network_layer(&buffer);
        s.info = buffer;
        to_physical_layer(&s);
        wait_for_event(&event);
}
```

➢接收方

- 完成一帧接收后
- 交给物理层一个哑帧
- 作为成功接收上一帧的确认

```
while (true) {
        wait_for_event(&event);
        from_physical_layer(&r);
        to_network_layer(&r.info);
        to_physical_layer(&s);
}
```

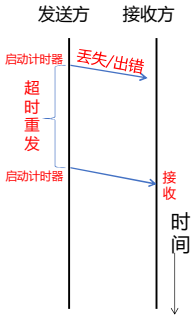

4. 有错信道上的单工停等式协议(1)

➢ 假设

- 通信信道可能会出错，导致：
  - 帧在传输过程中可能会被损坏，接收方能够检测出来
  - 帧在传输过程中可能会丢失，永远不可能到达接收方

➢ 一个简单的解决方案

- 发送方增加一个计时器(timer)，如果经过一段时间没有收到确认，发送方将超时，于是再次发送该帧



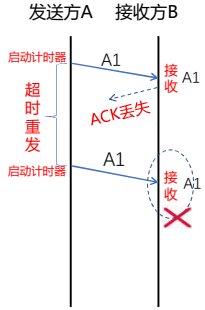
有错信道上的单工停等式协议(2)

➢ 考虑一个特别场景

- A发送帧A1
- B收到了A1
- B生成确认ACK
- ACK在传输中丢失
- A超时，重发A1
- B收到A1的另一个副本（并把它交给网络层）

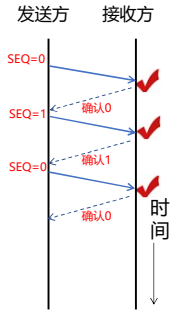
➢ 其他的场景

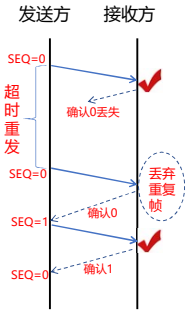
- 另一个导致副本产生的场景是过长的延时 (long delay)



有错信道上的单工停等式协议(3)

- 序号（SEQ：sequence number）
  - 接收方需要区分到达的帧是第一次发来的新帧
  - 让发送方在发送的帧的头部放一个序号，接收方可以检查它所收到的帧序号，由此判断这是个新帧还是应该被丢弃的重复帧。
- 序号所需要的最小位数（bits）
  - 在这个协议中，唯一不明确的地方出现在当前帧（序号m）和它的直接后续帧（序号m+1）
  - 1 bit序号(0或1)就足以满足要求。
- 自动重复请求，或带有重传的肯定确认
  - ARQ(Automatic Repeat reQuest)
  - PAR(Positive Acknowledgement with Retransmission)





有错信道上的单工停等式协议(4)

- 发送方
  - 初始化帧序号0，发送帧
  - 等待：正确的确认/错误的确认/超时
  - 正确确认：发送下一帧
  - 超时/错误确认：重发

```
next_frame_to_send = 0;
from_network_layer(&buffer);
while (true) {
        s.info = buffer;
        to_physical_layer(&s);
        start_timer(s.seq);
        wait_for_event(&event);
        if (event == frame_arrival){
                from_physical_layer(&s);
                if (s.ack == next_frame_to_send){
                        from_network_layer(&buffer);
                        inc(next_frame_to_send);
                }
        }
}
```

- 接收方
  - 初始化期待0号帧
  - 等待帧达到
  - 正确帧：交给网络层，并发送该帧确认
  - 错误帧：发送上一个成功接收帧的确认

```
frame_expected = 0;
while (true) {
        wait_for_event(&event);
        if (event == frame_arrival){
                from_physical_layer(&r);
                if (r.seq == frame_expected){
                        to_network_layer(&r.info);
                        inc(frame_expected);
                }
                s.ack = 1 - frame_expected;
                to_physical_layer(&s);
        }
}
```


有错信道上的单工停等式协议(5)

- 效率的评估
  - F = frame size (bits)
  - R = channel capacity (Bandwidth in bits/second)
  - I = propagation delay + processor service time (second)
  - 每帧发送时间 (Time to transmit a single frame) = F/R
  - 往返延迟 (Round Trip Delay) = D =2I
  - 停止等待协议的发送工作时间是F/R，空闲时间是D
  - 当 F/R<D 时：信道利用率 (line utilization)=F/(F+R·D) ＜ 50%

有错信道上的单工停等式协议(6)

- 停止等待协议的效率问题
  - 停止等待协议的问题是只能有一个没有被确认的帧在发送中
  - 举例: 长肥网络
    - Short frame length: 1000 bit frames
    - High bandwidth: 1 Mbps channel (卫星信道)
    - Long transit time: 270 ms propagation delay (传播延迟)

长肥网络 (LFN，Long Fat Network)：如果一个网络的带宽-延迟乘积 (bandwidth-delay product) 很明显的大于 10^5 bits (~12 kB)，则可以被认为是长肥网络

有错信道上的单工停等式协议(7)

- 信道利用率很低：
  - 每一帧的发送时间是 1毫秒（1000 bits/(1,000,000 bits/sec)）. 由于传播延迟较长，发送者在541毫秒之后才能收到确认，信道利用率1/541
- 一种提高效率的方法：使用更大的帧
  - 可以使用更大的帧，但是帧的最大长度受到信道比特错误率（BER，Bit Error Ratio）的限制，帧越大，在传输中出错的概率越高，将导致更多的重传

本章内容

3.1 数据链路层的设计问题
3.2 差错检测和纠正（P21）
3.3 基本的数据链路层协议（P53）
3.4 滑动窗口协议
3.5 数据链路协议实例（P99）

1. 停等协议的性能问题
2. 滑动窗口协议
3. 回退N协议(Go Back N)
4. 选择重传协议(SR)

1. 停等协议的性能问题

- 停止-等待机制降低了信道利用率
  - 设数据速率记为R，帧长度记为F，往返延迟记为D，则采用停-等协议的线路效率为：F/(F+R·D)
  - 假如将链路看成是一根管道，数据是管道中流动的水，那么在传输延迟较长的信道上，停-等协议无法使数据充满管道，因而信道利用率很低
- 解决办法
  - 流水线协议或管道协议：允许发送方在没收到确认前连续发送多个帧。

2. 滑动窗口协议—协议基本思想

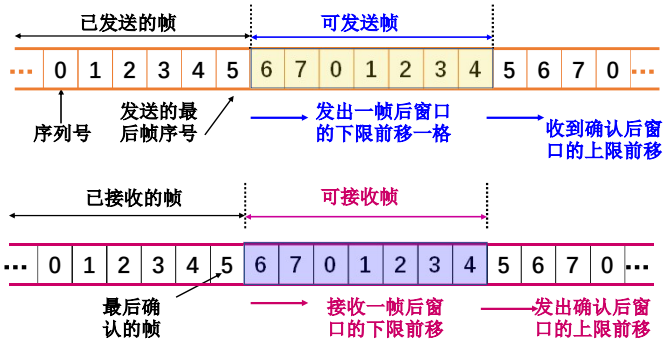
- 协议基本思想
  - 窗口机制
    - 发送方和接收方都具有一定容量的缓冲区（即窗口），发送端在收到确认之前可以发送多个帧
  - 问题
    - 发送端一次可以发送多少个帧?
    - 某个帧出错或丢失怎么处理?

滑动窗口协议—协议基本思想

- 目的
  - 对可以连续发出的最多帧数（已发出但未确认的帧）作限制
- 序号使用
  - 循环重复使用有限的帧序号
- 流量控制：接收窗口驱动发送窗口的转动
  - 发送窗口：其大小记作W_T，表示在收到对方确认的信息之前，可以连续发出的最多数据帧数
  - 接收窗口：其大小记作W_R，为可以连续接收的最多数据帧数
- 累计确认：不必对收到的分组逐个发送确认，而是对按序到达的最后一个分组发送确认

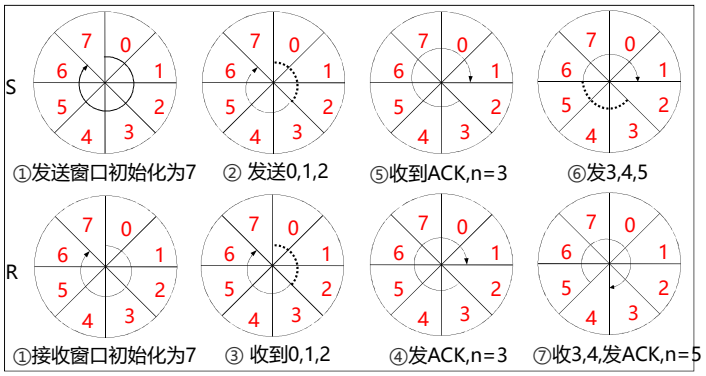
滑动窗口协议—协议基本思想

- 发送窗口与接收窗口(窗口大小7)



滑动窗口协议—协议基本思想

- 发送窗口与接收窗口(窗口大小7)



滑动窗口协议—协议的实现分析

发送方（W_T=W_R=1，序号空间0和1）

1. 初始化：ack_expected = frame_expected = next_frame_to_send = 0
2. 从网络层接收分组，放入相应的缓冲区，构造帧，物理层发送，开启计时。
3. 等待确认帧到达，从物理层接收一个帧，判断确认号是否正确，正确则停止计时器，并从网络层接收新分组。
4. 发送新的帧，跳转至3

```
while(1){
  wait_for_event(&event);
  if(event==frame_arrival) {
    from_physical_layer(&r);
    if(r.ack==next_frame_to_send) {
      from_network_layer(&buffer);
      inc(next_frame_to_send);
      }
    }
  s.info=buffer;
  s.seq=next_frame_to_send;
  s.ack=1-frame_expected;
  to_physical_layer(&s);
  start_timer(s.seq);
}
```

滑动窗口协议—协议的实现分析

接收方（$W_T=W_R=1$，序号空间0和1）

1. 初始化：ack_expected = frame_expected = next_frame_to_send = 0
2. 等待帧到达，从物理层接收一个帧，校验和计算，并判断收到的帧序号是否正确，正确则交给网络层处理，期待帧号增加。
3. 返回确认帧，跳转至2。

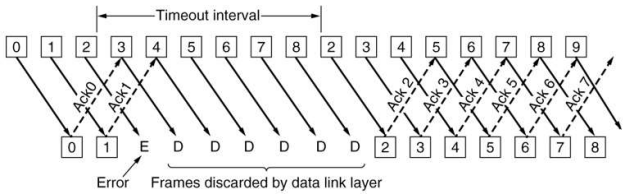
```
while(1){
  wait_for_event(&event);
  if(event==frame_arrival){
    from_physical_layer(&r);
    if(r.seq==frame_expected){
      to_network_layer(&r.info);
      inc(frame_expected);
      }
    }
  s.info=buffer;
  s.seq=next_frame_to_send;
  s.ack=1-frame_expected;
  to_physical_layer(&s);
  start_timer(s.seq);
}
```


3. 回退N协议(go back N)—协议设计思想

- 出错全部重发
  - 当接收端收到一个出错帧或乱序帧时，丢弃所有的后继帧，并且不为这些帧发送确认
  - 发送端超时后，重传所有未被确认的帧
- 适用场景
  - 该策略对应接收窗口为1的情况，即只能按顺序接收帧
- 优缺点
  - 优点：连续发送提高了信道利用率
  - 缺点：按序接收，出错后即便有正确帧到达也丢弃重传

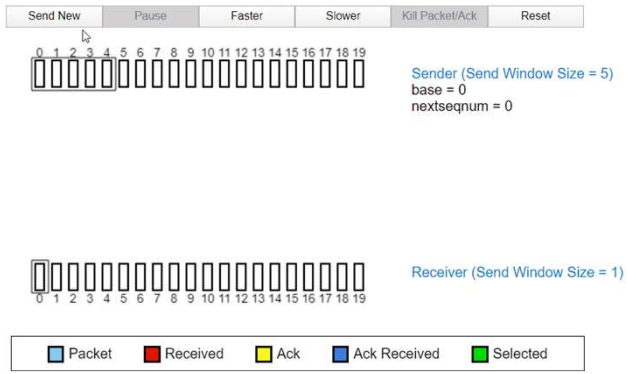
回退N协议—协议原理分析

- 基本原理
  - 当发送方发送了N个帧后，若发现该N帧的前一个帧在计时器超时后仍未返回其确认信息，则该帧被判为出错或丢失，此时发送方就重新发送出错帧及其后的N帧。
- 滑动窗口长度
  - 出错全部重发时，若帧序号为n位，接收窗口$W_R=1$，发送窗口 $W_T \le 2^n - 1$



回退N协议—协议原理分析

- 正常情况下的GBN

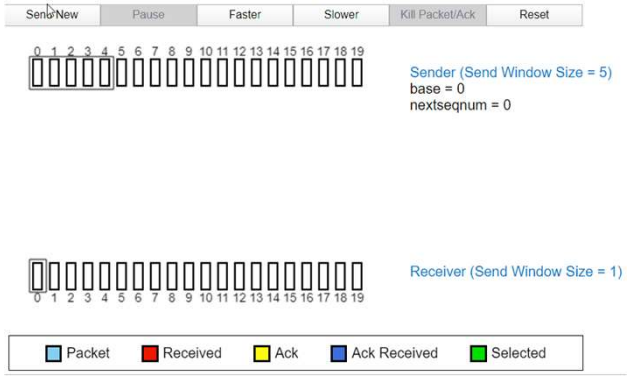


在回退N步协议中，允许发送方发送多个分组而不用等待确认

正常情况下，发送、接收窗口会随着帧的确认状态随包右移

回退N协议—协议原理分析

➢ 发送过程中数据包丢失

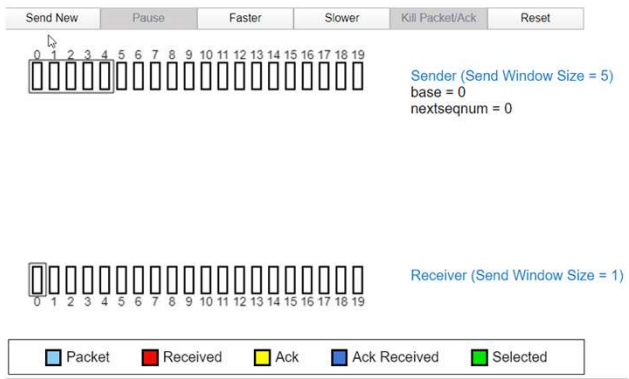


当发送过程中的2号包发生丢失时：

- 发送方没有收到接收方的ACK2，于是后面发送的ACK3, ACK4全部变成了ACK1，代表接收方因为丢失了分组2，所以分组3和分组4都被丢弃，全部返回ACK1
- 经过一段时间后，定时器确认超时没有收到ACK3,ACK4，所以发送方将重新发送。

回退N协议—协议原理分析

➢ 接收方返回的ACK丢失



假设ACK2发生了丢失时：

- 如果接收方没有收到分组2，则后面返回的都是ACK1，因为本次返回的为ACK3,ACK4，所以发送方可以判断接收方已经接收到消息，不再进行重复发送。

回退N协议—协议的实现分析

➢ 实现要点
- 必须增加序号范围、发送方需要缓存多个分组

➢ 两个窗口
- 发送窗口：发送方维持一组连续允许发送的帧序号
- 接收窗口：接收方维持一组连续的允许接收帧序号

➢ 发送方必须响应的三件事
- 上层的调用：检测有没有可以使用的序号，如果有就发送
- 收到ACK：对n号帧的确认采用累积确认的方式
- 超时事件：如果出现超时，就重传所有已发送未确认的分组

回退N协议—协议的实现分析

发送方：

1. 窗口尺寸：$1<W_T\leq 2^n-1$，最多连续发送窗口中的W_T个PDU
2. 窗口滑动：收到期望的ACK(k)：窗口底部移到PDU(k)，窗口顶部向前移动，始终保持窗口里有W_T个PDU未确认。
3. 窗口滑动后，发送新进入窗口的PDU
4. 超时重发： 超过T未收到期望的ACK，重发窗口中的PDU（回退整个窗口）
5. 超次数失败：超过最大重发次数N_{max}仍无正确应答

回退N协议—协议的实现分析

西南交通大学 Southwest Jiaotong University

接收方：

1. 窗口尺寸：W_R=1
2. 按序接收：按照PDU编号依序接收，出错、乱序PDU一律丢弃
3. 确认含义：ACK(k)表示对k-1及以前各编号的PDU的确认，同时期望接收第k号PDU
4. 确认策略：按序到达的PDU可立即确认，也可延迟确认(收到多帧后一起确认)，但出错或乱序的PDU，确认ACK(k)（期望接收k号PDU）或不应答

回退N协议—协议的实现分析

西南交通大学 Southwest Jiaotong University

实现基本过程如下：

1. 初始化。ack_expected = 0 (此时处于发送窗口的下沿)；next_frame_to_send = 0，frame_expected = 0 (初始化正在发送的帧和期待的帧序号)；nbuffered = 0 (进行发送大小初始化)
2. 等待事件发生（网络层准备好，帧到达，收到坏帧，超时）
3. 如果事件为网络层准备好，则执行以下步骤。从网络层接收一个分组，放入相应的缓冲区；发送窗口大小加1；使用缓冲区中的数据分组、next_frame_to_send和frame_expected构造帧，继续发送；next_frame_to_send加1；跳转（7）

```
while(1) {
  wait_for_event(&event);
  switch(event){
    case network_layer_ready:
      from_network_layer(&buffer[next_frame_to_send]);
      nbuffered=nbuffered+1;
      send_data(next_frame_to_send, frame_expected,
buffer);
      inc(next_frame_to_send);
      break;
```


回退N协议—协议的实现分析

西南交通大学 Southwest Jiaotong University

4. 如果事件为帧到达，则从物理层接收一个帧，则执行以下步骤。首先检查帧的seq域，若正是期待接收的帧（seq = frame_expected），将帧中携带的分组交给网络层，frame_expected加1；然后检查帧的ack域，若ack落于发送窗口内，表明该序号及其之前所有序号的帧均已正确收到，因此终止这些帧的计时器，修改发送窗口大小及发送窗口下沿值将这些帧去掉，继续执行步骤（7）

```
case frame_arrival:
   from_physical_layer(&r);
   if(r.seq==frame_expected) {
      to_network_layer(&r.info);
      inc(frame_expected);
      }
   while(between(ack_expected, r.ack, net_frame_to_send)){
      nbuffered=nbuffered-1;
      stop_timer(ack_expected);
      inc(ack_expected);
      }
   break;
```


回退N协议—协议的实现分析

西南交通大学 Southwest Jiaotong University

5. 如果事件是收到坏帧，继续执行步骤（7）
6. 如果事件是超时。使next_frame_to_send = ack_expected，从发生超时的帧开始重发发送窗口内的所有帧，然后继续执行步骤（7）
7. 若发送窗口大小小于所允许的最大值（MAX-SEQ），则可继续允许网络层发送，否则则暂停网络层发送

```
  case cksum_err: ;
     break;
  case timeout:
     next_frame_to_send=ack_expected;
     for(i=1; i<=nbuffered; i++){
        send_data(next_frame_to_send, frame_expected, buffer);
        inc(next_frame_to_send);
        }
  }
  if(nbuffered<MAX_SEQ)
     enable_network_layer();
  else
     disable_network_layer();
}
```

4. 选择重传协议(SR)—协议设计思想

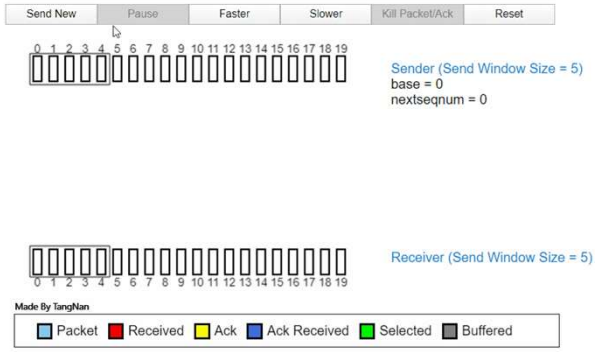
- 设计思想
  - 若发送方发出连续的若干帧后，收到对其中某一帧的否认帧，或某一帧的定时器超时， 则只重传该出错帧或计时器超时的数据帧
- 适用场景
  - 该策略对应接收窗口大于1的情况，即暂存接收窗口中序号在出错帧之后的数据帧
- 优缺点
  - 优点：避免重传已正确传送的帧
  - 缺点：在接收端需要占用一定容量的缓存

选择重传协议—协议原理分析

- 基本原理
  - 在发送过程中，如果一个数据帧计时器超时，就认为该帧丢失或者被破坏；接收端只把出错的的帧丢弃，其后面的数据帧保存在缓存中，并向发送端回复NAK；发送端接收到NAK时，只重传出错的帧
  - 如果落在窗口内的帧从未接受过，那么存储起来，等比它序列号小的所有帧都正确接收后，按次序交付给网络层
  - 接收端收到的数据包的顺序可能和发送的数据包顺序不一样，因此在数据包里必须含有顺序号来帮助接收端进行排序。
- 滑动窗口长度
  - 发送窗口的尺寸：$W_T \le 2^{n-1}$,发送窗口应等于或小于序号空间的一半

选择重传协议—协议原理分析

- 正常情况下SR

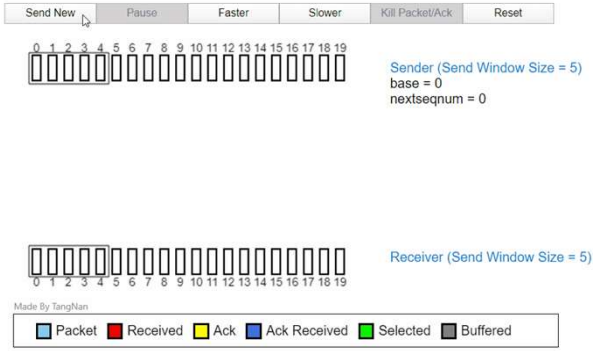


同样，在选择重传协议中，也允许发送方发送多个分组而不用等待确认，与回退N步不同的是，接收窗口大小是大于1的

正常情况下，发送、接收窗口会随着帧的确认状态随包右移

选择重传协议—协议原理分析

- 发送过程中发生丢包

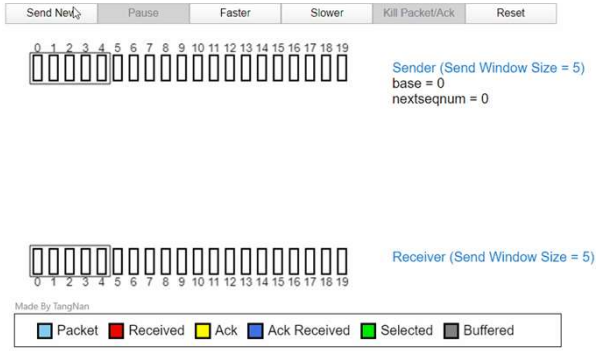


当发送过程中的2号包发生丢失时：

- 与GNB不同的是，接收方在没有收到分组2的情况下，依然返回了ACK3,ACK4
- 当ACK1返回以后，分组5，分组6就已经可以发送。然后在接收方，分组3456都已经被缓存，等待分组2的计时器超时后，分组2将重新发送。

选择重传协议—协议原理分析

➢ 确认过程发生丢包



当ACK2丢弃时：

- 发送方的分组已经由0-4到了2-6,在最后2-6的窗口中。
- 分组2会因为ACK2被丢失然后在计时器计时后重新发送一次。
- 如果在接收过程中有丢失发生，选择重传SR的效率是不如回退N步GBN的

选择重传协议—协议实现分析

➢ 实现要点

- 在SR中，和GBN不同，SR是给每一个PDU设置定时器，发送端只重传出错PDU。接收端在接收到乱序的PDU的时候会进行缓存，当前面的PDU到达以后一起提交给上层

➢ 两个窗口

- 发送窗口：发送方维持一组连续允许发送的帧序号
- 接收窗口：接收方维持一组连续的允许接收帧序号

➢ 发送方必须响应的三件事

- 上层的调用：检测有没有可以使用的序号，如果有就发送
- 收到ACK：如果收到的是最小序号的ACK，窗口滑动。如果收到其他序号的ACK，进行标记
- 超时事件：每个PDU都有定时器，哪个超时重传哪个

选择重传协议—协议实现分析

➢ 实现基本过程与回退N协议基本类似，其中

发送方：

1. 窗口尺寸：$1<W_T\leq 2^{n-1}$，最多连续发送窗口中的W_T个PDU
2. 窗口滑动：与回退N帧协议相同
3. 选择重发：收到NAK (k)，重发PDU (k)
4. 超时重发：超过T未收到期望的ACK，重发当前超时未应答的PDU
5. 超次数失败：超过最大重发次数N_{max}仍无正确应答，报告上层失败

选择重传协议—协议实现分析

接收方：

1. 窗口尺寸：$1<W_R\leq 2^{n-1}$
2. 窗口滑动：窗口底部数据上交，窗口向前滑动一步
3. 窗口内接收：窗口内的PDU全部接收，存储出错的后续PDU，按序交付；窗口外的PDU一律丢弃
4. 确认策略：按序到达的PDU可立即确认，也可延迟确认(收到多帧后一起确认)ACK (k)；出错用否定性确认NAK(k)（期望重发k号PDU）

选择重传协议—协议实现分析

实现基本过程如下：

1、初始化。ack_expected = 0 (此时处于发送窗口的下沿)；next_frame_to_send = 0，frame_expected = 0（初始化正在发送的帧和期待的帧序号）；nbuffered = 0（进行发送窗口大小初始化）；

2、等待事件发生（网络层准备好，帧到达，收到坏帧，超时，确认超时）。

3、如果事件为网络层准备好，则执行以下步骤。从网络层接收一个分组，放入相应的缓冲区；发送窗口大小加1；使用缓冲区中的数据分组、next_frame_to_send和frame_expected构造帧，继续发送；next_frame_to_send加1；跳转（8）；

4、如果事件为帧到达，则从物理层接收一个帧，则执行以下步骤。首先检查帧的kind域，若是数据包，再检查seq域，若不是期待接收的帧（seq！= frame_expected）并且不是nak，则发送nak，否则开启定时器；如果seq落入接收窗口之内并且没有被接收，则接收帧，将帧中携带的分组交给网络层，frame_expected、too_far加1,开启确认定时器；若kind为nak则重新发送数据。最后检查帧的ack域，若ack落于发送窗口内，表明该序号及其之前所有序号的帧均已正确收到，因此终止这些帧的计时器，修改发送窗口大小及发送窗口下沿值将这些帧去掉，继续执行步骤（8）；

5、如果事件是收到坏帧，如果no_nak为真，则发送nak帧，然后继续执行步骤（8）。

6、如果事件是发送超时，即：next_frame_to_send = oldest_frame，则重发超时帧，然后继续执行步骤（8）。

7、如果事件是确认超时，则重发超时的确认帧，然后继续执行步骤（8）。

8、若发送窗口大小小于所允许的最大值（MAX_SEQ），则可继续向网络层发送，否则则暂停继续向网络层发送，同时返回互步骤（2）等待。

选择重传协议—协议的实现分析

核心代码

```
while(1) {
  wait_for_event(&event);   /* 包括5种情况 */
  switch(event) {
    case network_layer_ready:  /* 从网络层接收数据，传输新帧 */
      from_network_layer(&out_buffer[next_frame_to_send%NR_BUFS]);
      nbuffered=nbuffered+1;
      send_frame(data,next_frame_to_send, frame_expected, out_buffer);
      inc(next_frame_to_send);
      break;
   case frame_arrival:  /* 数据帧或控制帧到达 */
      from_physical_layer(&r);
      if (r.kind==data) {
        if ((r.seq!=frame_expected) && no_nak)
          send_frame(nak, 0, frame_expected, out_buf);  else  start_ack_timer();
        if (between(frame_expected, r.seq,too_far) &&arrived[r.seq%NR_BUFS]==false)) {
          arrived[r.seq%NR_BUFS]=true;
          in_buf[r.seq%NR_BUFS]=r.info;
          while(arrived[frame_expected%NR_BUFS]){
            to_network_layer(&in_buf[frame_expected%NR_BUFS]);
            no_nak=true;
            arrived[frame_expected%NR_BUFS]=false;
```


选择重传协议—协议的实现分析

核心代码

```
            inc(frame_expected);
            inc(too_far);
            start_ack_timer();
            }
          }
        }
      if((r.kind==nak) &&  between(ack_expected ,(r.ack+1)%(MAX_SEQ+1), next_frame_to_send))
          send_frame(data,(r.ack+1)%(MAX_SEQ+1),frame_expected, out_buf);
      while(between(ack_expected, r.ack, next_frame_to_send)) {
          nbuffered=nbuffered-1;
          stop_timer(ack_expected%NR_BUFS);
          inc(ack_expected);
          }
      break;
   case cksum_err: if(no_nak) send_frame(nak, 0, frame_expected, out_buf); break;  /* 帧出错 */
   case timeout: send_frame(data, oldest_frame,frame_expected,out_buf); break;  /* 超时 */
   case ack_timeout: send_frame(ack, 0, frame_expected, out_buf); break;  /* ACK定时器超时，发送ACK */
   }
 if (nbuffered<MAX_BUFS)  enable_network_layer();  else  disable_network_layer();
 }
```


本章内容

3.1 数据链路层的设计问题
3.2 差错检测和纠正（P21）
3.3 基本的数据链路层协议（P53）
3.4 滑动窗口协议（P69）
3.5 数据链路协议实例
1. 点到点链路层协议PPP
2. PPPoE

1. PPP协议简介

- PPP (Point-to-Point Protocol)协议由IETF制定，1994年成为正式标准（RFC1661）
- PPP协议是目前使用最多的数据链路层协议之一
- 能够在不同的链路上运行
- 能够承载不同的网络层分组
- 特点：简单、灵活

PPP协议实现的功能

- 利用帧定界符封装成帧
- 填充技术实现透明数据传输：字节填充、零比特填充
- 帧的差错检测
- 实时监测链路工作状态
- 设置链路最大传输单元（MTU）
- 网络层地址协商机制
- 数据压缩协商机制

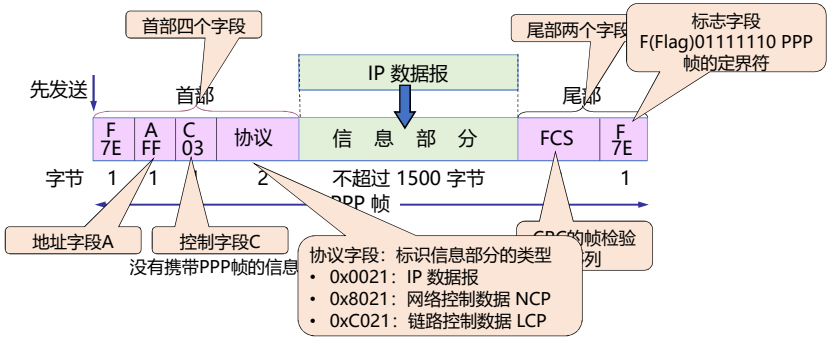
PPP协议未实现的功能

- 帧数据的纠错功能
  - 数据链路层的PPP协议只进行检错，PPP协议是不可靠传输协议
- 流量控制功能
  - PPP协议未实现点到点的流量控制
- 可靠传输功能
  - PPP为不可靠协议，不使用帧的序号。不可靠网络中可能使用有序号的工作方式。
- 多点连接功能
  - PPP协议不支持多点线路，只支持点对点的链路通信。
- 单工和半双工链路
  - PPP协议支持全双工链路

PPP协议的构成

- 封装 (Encapsulation)
  - 提供在同一链路上支持不同的网络层协议
  - PPP既支持异步链路（无奇偶检验的8比特数据），也支持面向比特的同步链路
  - IP数据包在PPP帧中是其信息部分，其长度受到MTU的限制
- 链路控制协议 LCP (Link Control Protocol)。
  - 用来建立、配置和测试数据链路的链路控制协议，通信双方可协商一些选项
- 网络控制协议 NCP (Network Control Protocol)。
  - 其中每个协议支持一种不同的网络层协议，如IP、OSI的网络层、DECnet、AppleTalk等

PPP协议的帧格式



PPP协议的帧格式

➢ PPP协议中的透明传输

- 当 PPP 用在异步传输时，使用一种特殊的字符填充法
- 当 PPP 用在同步传输链路时，协议规定采用硬件来完成比特填充（与 HDLC 的做法类似）

PPP协议的帧格式

➢ 字节填充

- 避免在信息字段中出现和标志字段一样的比特组合（0X7E）
- 当PPP使用异步传输时，定义转义字符0X7D，并使用字节填充
  - 将信息字段中出现的0x7E 字节转变成为2字节序列(0x7D, 0x5E)
  - 若信息字段中出现一个0x7D 字节, 则将其转变成为2字节序列(0x7D, 0x5D)
  - 若信息字段中出现 ASCII 码的控制字符（即数值小于 0x20 的字符），则在该字符前面要加入一个 0x7D 字节，同时将该字符的编码加以改变
- 在发送端进行了字节填充，链路上传递的信息字节数超过原来的信息字节数。接收端进行字节填充相反的工作，恢复为原来的信息

PPP协议的帧格式

➢ 字节填充

- 0x7e 字节填充后编码为 0x7d, 0x5e. (Flag Sequence)帧序列
- 0x7d 字节填充后编码为 0x7d, 0x5d. (Control Escape)控制转义符
- 0x03 字节填充后编码为 0x7d, 0x23. (ETX)文本结束符
- 0x11 字节填充后编码为 0x7d, 0x31. (XON)重新传输
- 0x13 字节填充后编码为 0x7d, 0x33. (XOFF)传输暂停
- 0x91 字节填充后编码为 0x7d, 0xb1. (XON with parity set)带奇偶校验集合的XON
- 0x93 字节填充后编码为 0x7d, 0xb3. (XOFF with parity set)带奇偶校验集合的XOFF

PPP协议的帧格式

- 零比特填充
  - PPP协议用在SONET/SDH链路时，使用同步传输（一连串的比特连续传送），不是异步传输（逐个字符地传送）
    - 采用比特填充方法来实现透明传输。
    - 信息字段中出现了和标志字段 F 完全一样的 8 比特组合（0x7E）；
    - 发送端在 5 个连 1 之后填入 0 比特再发送出去；
    - 在接收端把 5 个连 1之后的 0 比特删除

PPP协议的帧格式

- 零比特填充

1. 信息字段中出现了和标志字段 F 完全一样的 8 比特组合

0 1 0 0 1 1 1 1 1 1 0 0 0 1 0 1 0

会被误认为是标志字段 F

2. 发送端在 5 个连 1 之后填入 0 比特再发送出去

0 1 0 0 1 1 1 1 1 0 1 0 0 0 1 0 1 0

发送端填入 0 比特

3. 在接收端把 5 个连 1之后的 0 比特删除

0 1 0 0 1 1 1 1 1 0 1 0 0 0 1 0 1 0

接收端删除填入的 0 比特

PPP协议的工作状态及转换

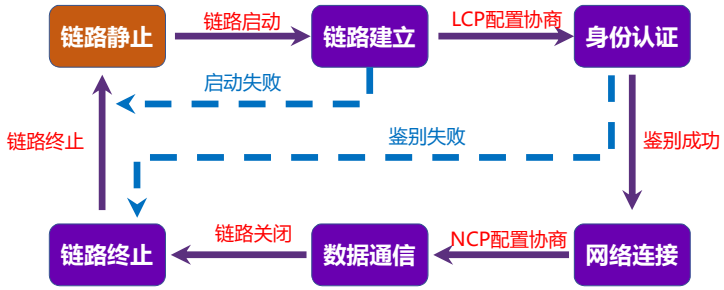
- 在点对点链路上建立通信，通信双方通过LCP报文进行配置和测试链路
- 对等实体进行身份认证（可选）
- PPP协议必须发送NCP报文来选择和配置一个或多个网络层协议
- 双方网络层协议可以通过PPP链路发送数据报，并持续通信
- LCP或NCP数据报显式关闭链路，或者异常事件导致关闭链路

PPP协议的工作状态及转换

- 状态转换图(State Transform Diagram)
  - 状态转换图通过描述系统的状态和引起系统状态转换的事件，来表示系统的行为，指出作为特定事件的结果将执行哪些动作。描述系统对内部事件和外部事件如何响应，如何动作。
  - 状态转换图中以节点表示状态，有向边来表示内外部事件和响应。
  - 本节将采用状态转换图来表示PPP协议工作的行为模型

- 状态
  - PPP协议的状态转换图中共设置了六个状态：
  - 链路静止、链路建立、身份认证、网络连接、数据通信、链路终止
- 状态的变迁
  - PPP协议中通过一些外部事件来触发状态的变迁，如：
  - 链路启动、LCP配置协商、鉴别成功、NCP配置协商、链路关闭、链路终止等

PPP协议的工作状态及转换

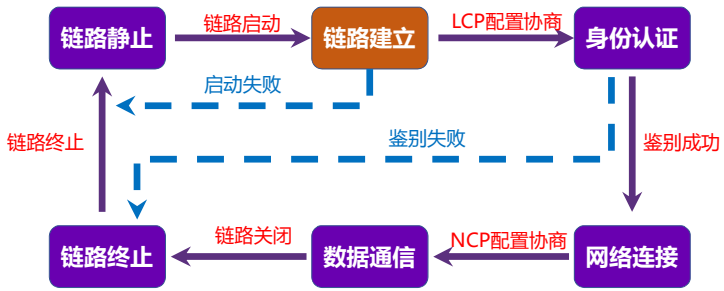


PPP协议的工作状态及转换

➢ 链路静止状态

- PPP链路的起始和终止状态，通信双方尚无链路
- 当外部事件表明物理层可以使用时，PPP将进入链路建立阶段。
  - 如载波检测、网络管理员配置
  - PC机通过调制解调器呼叫路由器时，路由器能检测到调制解调器发出的载波信号，并作出应答。

PPP协议的工作状态及转换



PPP协议的工作状态及转换

➢ 链路建立状态

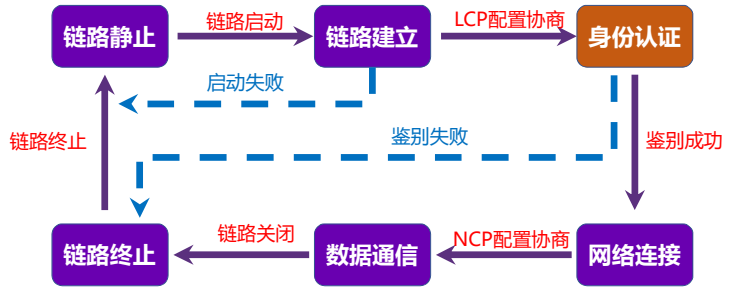
- LCP通过交换配置包来建立连接
- LCP配置选项包括：链路上的最大帧长、所使用的鉴别协议的规约、不使用PPP帧中的地址和控制字段
- LCP开始协商一些配置选项，发送LCP的配置请求帧（Configure-Request），协议字段为LCP对应的代码，信息字段包含特定的配置请求
- 链路的另一端可以发送几种响应的一种
  - 配置确认帧（Configure-Ack）：所有选项都接受
  - 配置否认帧（Configure-Nak）：所有选项都理解但是不能接受
  - 配置拒绝帧（Configure-Reject）：选项有的无法识别或不能接受，需要协商

PPP协议的工作状态及转换

➢ 链路建立状态

- LCP帧共有11种（以上为4种配置型帧）。
  - 关闭线路：Terminate-request请求关闭线路帧，Terminate-ack接受关闭线路；
  - 应答方无法理解：Code-reject收到了未知的请求，Protocol-reject被请求的协议未知； 可能是传输错误或者LCP协议版本不同
  - 用于测试线路质量：Echo-request回送请求帧，Echo-reply回送应答帧；
  - 用于测试：Discard-request只需丢弃该帧

PPP协议的工作状态及转换

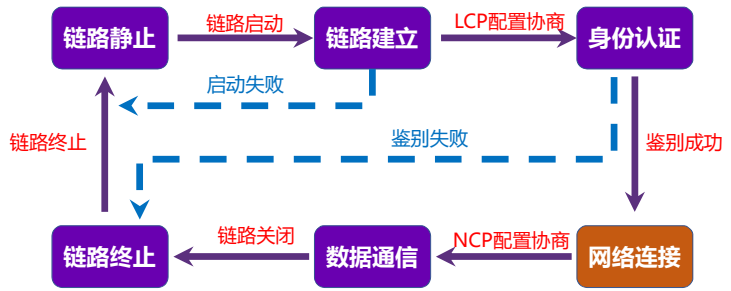


PPP协议的工作状态及转换

➢ 身份认证状态

- 在允许交换网络层协议包之前，对等点可选身份验证
- 默认情况下，不需要身份验证
- 如果选择身份验证协议，需要在链接建立阶段请求使用该身份验证协议
- 认证将在链接建立后进行，链路质量确认可能会同时发生
- 在身份验证完成之前，不能从身份验证阶段推进到网络层协议阶段
- 如果身份验证失败，则身份验证器应该继续到链接终止阶段
- 此阶段只允许链路控制协议、认证协议和链路质量监控包
- 此阶段接收到的所有其他数据包必须丢弃

PPP协议的工作状态及转换

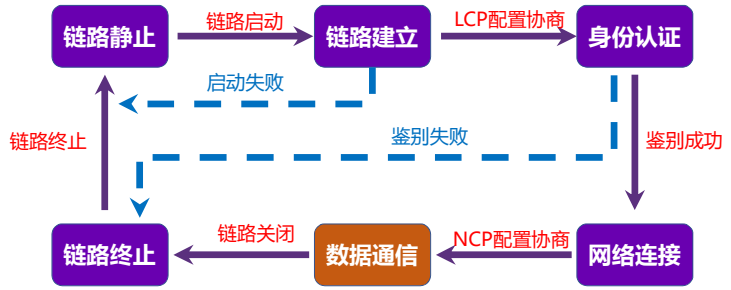


PPP协议的工作状态及转换

➢ 网络连接状态

- 完成了前面的阶段，每个网络层协议(如IP、IPX或AppleTalk)必须由适当的网络控制协议(NCP)单独配置
- 每个NCP可以在任何时候打开和关闭
- NCP到达打开状态后，PPP将携带相应的网络层协议包
- 如果相应的NCP不处于打开状态时，接收到的任何受支持的网络层协议包都必须丢弃
- 在此阶段，链路流量由LCP、NCP和网络层协议包的任何可能组合组成

PPP协议的工作状态及转换

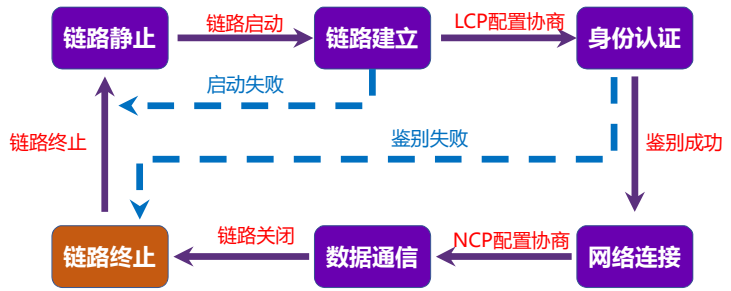


PPP协议的工作状态及转换

➢ 数据通信状态（网络打开状态）

- 链路的两个PPP端点可以彼此向对方发送分组
- 或者发送检查链路状态的Echo-*帧

PPP协议的工作状态及转换



PPP协议的工作状态及转换

- PPP可以在任何时候终止链接。
  - 运营商丢失、身份验证失败、链接质量失败、空闲期计时器过期或链接的管理关闭造成的
- LCP用于通过交换终止包来关闭链接。
  - 当连接关闭时，PPP通知网络层协议
  - 在交换终止包之后，实现应该向物理层发出断开的信号，以强制终止链接，特别是在身份验证失败的情况下
- Terminate-Request的发送方在收到Terminate-Ack或重启计数器过期后断开连接。
- Terminate-Request的接收方等待对等方断开连接，并且在发送Terminate-Ack之后，在至少一次重新启动时间过去之前，绝不能断开连接。
- PPP随后进入链路静止阶段。
- 在此阶段接收到的任何非LCP数据包都必须丢弃。

本章内容

3.1 数据链路层的设计问题

3.2 差错检测和纠正

3.3 基本的数据链路层协议

3.4 滑动窗口协议

3.5 数据链路协议实例

1. 点到点链路层协议PPP
2. PPPoE

实验报告：1）内容；2）格式；3）递交时间

2. PPPoE概述

- Ethernet优点
  - 原理简单，应用非常广，设备成本低
- Ethernet缺点
  - 安全性较低、不宜管理：使用广播信道，造成了安全性较低，无认证功能
- PPP优点
  - 原理简单
  - 安全性高：点对点信道，提供认证机制
  - 提供良好的访问控制和计费功能

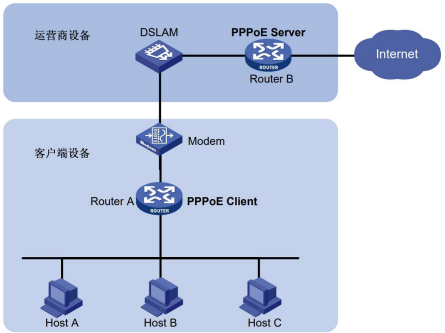
PPPoE概述

- PPPoE（Point-to-Point Protocol over Ethernet）
  - 提供在以太网链路上的PPP连接
  - 实现了传统以太网不能提供的身份验证、加密，以及压缩等功能
  - 实现基于用户的访问控制、计费、业务类型分类等，运营商广泛支持
  - PPPoE使用Client/Server模型，服务器通常是接入服务器

PPPoE组网方式

➢ 组网方式1

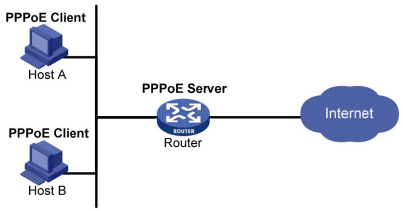
- 设备之间建立 PPP 会话，所有主机通过同一个 PPP 会话传送数据，主机上不用安装 PPPoE 客户端拨号软件，一般是一个机构共用一个账号
- PPPoE Client 位于机构内
- PPPoE Server 是运营商的设备



PPPoE组网方式

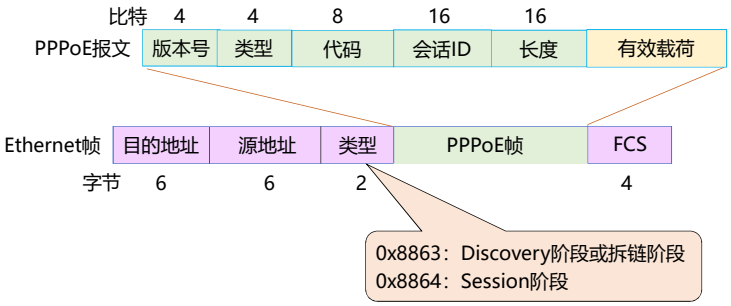
➢ 组网方式2

- PPP 会话建立在 主机和运营商的路由器之间，为每一个主机建立一个 PPP 会话，每个主机都是 PPPoE Client，每个主机有一个帐号，方便运营商对用户进行计费和控制
- 主机上需要安装PPPoE 客户端软件



PPPoE报文格式

➢ PPPoE报文格式及Ethernet帧封装



PPPoE报文格式

➢ 版本号：PPPoE 版本号，值为 0x1

➢ 类型：PPPoE 类型，值为 0x1

➢ 代码：PPPoE 报文类型

➢ 会话ID：

- 分配的PPPoE会话ID
- 与以太源地址和目的地址一起定义了一个会话

➢ 长度：定义有效载荷长度

代码值	PPPoE 报文类型
0x00	会话数据
0x09	PADI 报文
0x07	PADO 或 PADT 报文
0x19	PADR 报文
0x65	PADS 报文

PPPoE工作过程

- PPPoE可分为三个阶段：
  - Discovery阶段
    - 获取对方以太网地址，确定PPPoE会话ID
  - Session阶段
    - PPP协商阶段
    - PPP报数数据传输
  - Terminate阶段
    - 会话建立以后的任意时刻，发送报文结束会话

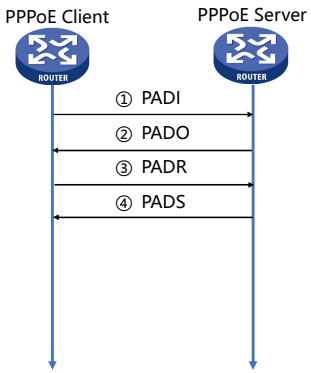
PPPoE工作过程

- Discovery阶段和Terminate阶段使用的报文类型

类型	描述
PADI（PPPoE Active Discovery Initiation）	PPPoE发现初始报文
PADO（PPPoE Active Discovery Offer）	PPPoE发现提供报文
PADR（PPPoE Active Discovery Request）	PPPoE发现请求报文
PADS（PPPoE Active Discovery Session-confirmation）	PPPoE发现会话确定报文
PADT（PPPoE Active Discovery Terminate）	PPPoE发现终止报文

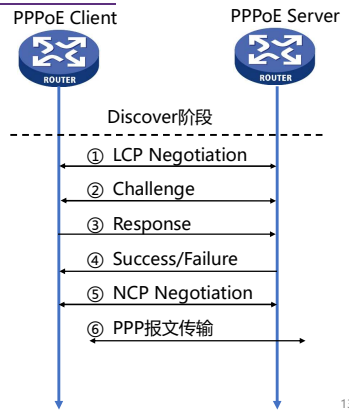
PPPoE工作过程

- Discovery阶段：
  - PPPoE Client广播PADI报文，报文中包含PPPoE Client想要得到的服务类型信息
  - 所有的PPPoE Server收到PADI报文之后，将其中请求的服务与自己能够提供的服务进行比较，若可提供，则单播回复PADO报文
  - 根据网络拓扑，PPPoE Client可能收到多个PPPoE Server发送的PADO报文，PPPoE Client选择最先收到PADO对应的 PPPoE Server，并单播PADR报文
  - PPPoE Server 产生唯一的Session ID，标识与 PPPoE Client 的会话，发送PADS报文把会话ID发送给PPPoE Client，进入Session阶段



PPPoE工作过程

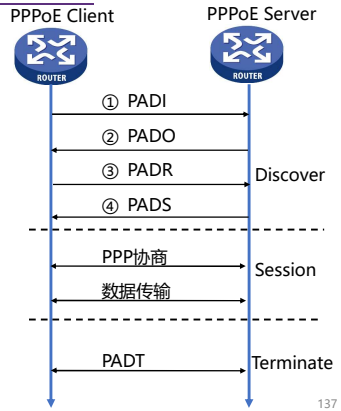
- Session阶段：
  - PPP协商阶段
    - 与普通的PPP协商方式一致，分为LCP、认证、NCP三个阶段
    - LCP 阶段主要完成建立、配置和检测链路连接。LCP 协商成功后，开始进行认证工作，认证协议类型由 LCP 协商结果决定
    - 认证成功后，进入NCP阶段。NCP是一个协议族，用于配置不同的网络层协议，常用的是IP控制协议(IPCP)，负责配置用户的IP和DNS等工作
  - PPP报文传输阶段



PPPoE工作过程

➢ Terminate阶段，终止方式有两种：

- PPP协议自身（比如PPP终结报文）来结束PPPoE会话
- 使用PADT报文终结会话
  - 进入PPPoE Session阶段后，PPPoE Client和PPPoE Server都可以通过发PADT报文的方式来结束PPPoE连接
  - PADT数据包可以在会话建立以后的任意时刻单播发送。



本章总结

➢ 通过本章学习，重点掌握如下知识点：

- 理解数据链路层在五层网络体系结构中的位置及解决的问题，了解数据链路层提供的服务类型
- 掌握差错检测和纠正的基本原理和典型的编码方法
- 掌握无错信道和有错信道上停等协议的设计和实现方法
- 理解停等协议的性能问题及滑动窗口协议的基本思想
- 掌握回退N和选择重传两种典型滑动窗口协议的工作机制
- 掌握点到点链路层协议PPP与PPPoE的工作机制